Dynamically Scheduling Deadline-Constrained Interleaved Workflows on Heterogeneous Computing Systems

Kun Cai, Quanwang Wu, *Senior Member, IEEE*, Mengchu Zhou, *Fellow, IEEE*, Chao Chen, *Senior Member, IEEE*, Junhao Wen, and Shouguang Wang

***Abstract*—Heterogeneous computing systems are extensively utilized to execute a wide range of time-critical services, which encompass numerous interdependent tasks organized in the form of workflows. In practice, the dynamic arrival of workflows often interleaves with their execution, leading to resource contention among multiple workflows and potentially causing QoS (Quality of Service) degradation. However, compared to the extensive research on single workflow scheduling, interleaved workflow scheduling has received relatively less attention. Moreover, the challenge of effectively scheduling limited computing resources to promptly complete consecutively arriving workflows remains underexplored, despite its practical importance. To fill this gap, this work proposes a method called Urgency-based List Scheduling (ULS) for dynamically scheduling deadline-constrained interleaved workflows. In ULS, a novel task property called urgency is introduced to prioritize tasks from multiple workflows by capturing real-time execution information, and each newly arrived workflow is scheduled with the outstanding tasks of prior workflows based on a list-based strategy to make more informed decisions. Extensive evaluation experiments are performed and the findings illustrate that ULS can achieve a reduction of at least 68% in deadline miss rates and 77% in overall tardiness compared to existing methods.**

***Index Terms*—Deadline constraint, heterogeneous computing systems, interleaved workflows, list scheduling, Quality of Service (QoS).**

I. Introduction

A HETEROGENEOUS computing system (HCS) is regarded as the backbone of many modern applications, e.g., Internet of Things, cloud computing and edge computing [1], [2]. They integrate the advantages of various processing elements (PEs), such as general-purpose processors and hardware accelerators, thereby providing high-desired Quality of Service (QoS) and cost-efficiency that homogeneous computing systems cannot achieve.

To represent concurrent applications deployed on such systems, a workflow model is extensively used and abstracted into a Direct Acyclic Graph (DAG), where nodes represent tasks and edges represent dependencies among them [3]. Workflow scheduling aims to allocate each task in the workflow to a certain time slice of a PE for execution, so as to meet user-specified QoS criteria. When used for time-critical applications, HCS should respond to external stimuli within a stipulated upper bound on time referred to as a deadline [4]. For example, a crucial QoS metric for video streaming is to minimize the deadline miss rate of transcoding tasks [5].

The workflow scheduling problem on both homogeneous and heterogeneous platforms falls into the category of strongly NP-hard problems. It has been a hot research topic in the services computing community for some decades and a great number of approaches have been proposed to address it by using low-overhead heuristics to yield fast but QoS-acceptable schedules [6]. Nevertheless, most existing efforts focus on scheduling single workflows, and these approaches cannot be directly applied to dynamic scenarios in practice, where multiple workflows arrive and execute interleavedly, and their scheduling outcomes mutually influence each other.

A few studies have been proposed to schedule multiple workflows in HCS [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17]. Some of these studies assume that workflows arrive simultaneously [7], [8], [9] or periodically [10], [11], [12]. Such strong assumptions on the workflow arrival pattern may only hold in some specific scenarios. In [13], [14], [15], [16], [17], no assumption is set and multiple workflows arrive arbitrarily. Workflows are scheduled independently in [13], [14], [15], [16], but experimental results from [17], [18] suggest that holistic workflow scheduling can yield superior performance. The work [17] aims to minimize the execution time of each workflow. However, it does not differentiate priorities among workflows and cannot optimize their deadline satisfaction rate. How to co-schedule interleaved workflows on HCS to meet their respective deadlines remains an unresolved issue despite its practical importance.

In this work, we propose an effective dynamic scheduling approach with awareness of deadlines to address this open issue.

Received 30 June 2024; revised 12 December 2024; accepted 12 January 2025. Date of publication 30 January 2025; date of current version 10 April 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62172065, Grant U22A2026 and Grant 62322601. *(Corresponding author: Quanwang Wu.)*

Kun Cai, Quanwang Wu, and Chao Chen are with the School of Computer Science, Chongqing University, Chongqing 400044, China (e-mail: caik@cqu.edu.cn; wqw@cqu.edu.cn; cschenchao@cqu.edu.cn).

Mengchu Zhou is with the Department of Electrical and Computer Engineering, New Jersey Institute of Technology, Newark, NJ 07102 USA (e-mail: zhou@njit.edu).

Junhao Wen is with the School of Big Data and Software Engineering, Chongqing University, Chongqing 400044, China (e-mail: jhwen@cqu.edu.cn).

Shouguang Wang is with the School of Information and Electronic Engineering, Zhejiang Gongshang University, Hangzhou 310018, China (e-mail: wsg5000@hotmail.com).

Digital Object Identifier 10.1109/TSC.2025.3536317

1939-1374 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Zhejiang University of Finance & Economics. Downloaded on August 01,2025 at 02:24:46 UTC from IEEE Xplore. Restrictions apply.

The novel contributions that this work aims to make are as follows:

1) It, for the first time, proposes a model for co-scheduling deadline-constrained interleaved workflows, which arrive dynamically and are executed on HCS, aiming to minimize deadline miss rate (DMR) and overall tardiness (OT).
2) To minimize DMR and OT, it devises an Urgency-based List Scheduling (ULS) heuristic to schedule each newly injected workflow as well as the outstanding tasks of prior workflows. It introduces a task property called urgency to prioritize tasks from multiple workflows based on real-time interleaved workflow execution analysis, and employs a list-based strategy to schedule them.
3) It conducts an extensive evaluation of the proposed algorithm on simulated HCS platforms. Dynamically interleaved workload mixtures are used and experimental results verify the superiority of the proposed algorithm over the state-of-the-art methods.

Section II reviews related work in literature. Section III formulates a scheduling model while Section IV presents our proposed algorithm. Section V provides experimental results and their analysis. Finally, Section VI concludes this article.

II. Related Work

In this section, we first briefly review literature on single workflow scheduling, and then discuss related work on multi-workflow scheduling.

A. Single Workflow Scheduling

Most of the existing research efforts on workflow scheduling have been devoted to the scheduling of a single workflow, typically with the objective of minimizing its makespan. In other words, they assume that all candidate computation resources are used for one specific workflow during its execution, and no other workflows arrive to contend for the resources. Classic algorithms for makespan minimization include list scheduling, clustering, duplication, and meta-heuristics [6]. Among them, list scheduling is the most widely-used heuristic. Its representatives are Heterogeneous Earliest Finish Time (HEFT) [19] and Predict Earliest Finish Time (PEFT) [20]. It consists of two main steps: construct a scheduling list (i.e., a sequence of tasks) by assigning task priorities and then sequentially allocate each task from this list to the resource which allows for the earliest finish time. The clustering heuristic combines tasks into clusters and allocates clusters of tasks as a whole to computation resources to avoid data transfer between tasks in the same cluster [21], [22]. The duplication heuristic aims to properly duplicate tasks on the same resource with the target task to reduce workflow makespan [23]. Besides, meta-heuristics are also used for workflow scheduling, such as genetic algorithm, firefly algorithm and genetic programming [24], [25], [26].

B. Multi-workflow Scheduling

It is quite a common scenario to schedule multiple workflows on HCS in reality and this problem is more challenging than a single workflow scheduling problem. Its research is relatively less than the latter's [27], [28], [29]. According to workflow arrival patterns, its related studies can be divided into three types: simultaneous [7], [8], [9], periodical [10], [11], [12] and arbitrary [13], [14], [17], [18]. For the latter two patterns, the arrival and execution of multiple workflows may be interleaved, leading to mutual influence on their scheduling outcomes.

For the first pattern, multiple workflows all arrive at the same time instant. For example, in [7], Wang et al. focus on how to schedule multiple workflows with different deadlines on heterogeneous cloud computing resources based on coalition reinforcement learning so that the resource utilization rate is optimized. Similarly, in [8], a multiple workflow scheduling method is proposed to minimize total makespan and cost under deadline constraints. In [9], Malawski et al. propose scheduling algorithms for cost- and deadline-constrained workflow ensembles in heterogeneous cloud resources.

For the second one, multiple workflows are assumed to commence execution in the beginning and continue to execute periodically. In [10], a list-based energy-aware scheduler is proposed to minimize aggregate energy dissipated over the schedule length for one hyper-period. Particularly, it first merges all independent workflows into one. Roy et al. also focus on scheduling a set of periodic real-time workflows on HCS [11]. A three-phase list-based hierarchical scheduling method is proposed to optimize dynamic energy dissipation under deadline constraints. Taheri et al. propose a cloud broker for executing deadline-constrained periodic workflows [12]. It adopts a hybrid scheduling method and uses heterogeneous cloud resources to minimize the monetary cost. Zhao et al. propose a multi-DAG scheduling approach that models both parallelism and dependencies [30]. Assuming homogeneous computational resources and neglecting communication delays among tasks, they derive a response time upper bound by using parallelism-aware analysis and formal proofs.

For the last one, no assumption is set and multiple workflows may arrive at any time. Since the timing of interleaving workflows is unpredictable, determining how to conduct effective scheduling becomes more challenging. Bittencourt and Madeira design four different strategies for scheduling multiple workflows on homogeneous grid computing resources and evaluate them with regard to makespan and fairness [18]. To simulate workflows arriving arbitrarily in experiments, a Poisson distribution is used. Chen et al. define the risk of running a job on HCS regarding hardware Trojan misdetection [13]. Static and dynamic scheduling strategies are developed to minimize job-level risk as well as deadline violation rate by deriving the task-level sub-deadline. For both strategies, workflows are scheduled independently even when they are interleaved. In [14], Ma et al. present a three-stage method to schedule multiple workflows separately on the cloud to minimize usage cost under deadline constraints. Mack et al. focus on performant scheduling of highly interleaved workflows on HCS [17]. A family of scheduling algorithms is proposed based on HEFT. Among them, HEFT_{RT} is a runtime variant of HEFT with DAG merging and dynamic dependencies, whereas $\text{HEFT}_{\text{EDP-LB}}$ is a load-balanced one that better balances energy consumption

Authorized licensed use limited to: Zhejiang University of Finance & Economics. Downloaded on August 01,2025 at 02:24:46 UTC from IEEE Xplore. Restrictions apply.

TABLE I
A TAXONOMY OF RELATED WORKS ON MULTI-WORKFLOW SCHEDULING

Method	Arrival Pattern	Scheduling Type	Multi-Workflow Handling Manner	Objective
[7]	Simultaneous	Static	Holistic	Makespan, Rental Cost
[8]	Simultaneous	Static	Holistic	Makespan, Rental Cost
[9]	Simultaneous	Static	Holistic	No. of Completed Workflows within Deadline and Budget
[10]	Periodical	Static	Holistic	Deadline Miss Rate, Energy Consumption
[11]	Periodical	Static	Holistic	Deadline Miss Rate, Energy Consumption
[12]	Periodical	Dynamic	Holistic	Deadline-constrained Rental Cost
[13]	Arbitrary	Static & Dynamic	Independent	Deadline Miss Rate, Job-Level Risk
[14]	Arbitrary	Dynamic	Independent	Deadline-constrained Rental Cost
[15]	Arbitrary	Dynamic	Independent	Deadline Miss Rate, Average Response Time
[16]	Arbitrary	Dynamic	Independent	Deadline Miss Rate, Rental Cost
[17]	Arbitrary	Dynamic	Holistic	Makespan, Energy Consumption
[18]	Arbitrary	Dynamic	Independent & Holistic	Makespan, Fairness
This Paper	Arbitrary	Dynamic	Holistic	Deadline Miss Rate, Overall Tardiness

TABLE II
KEY NOTATIONS

Category	Notation	Description
Workflow	$\mathcal{G}$	the set of all considered workflows
	G_k	the k-th workflow, $G_k \in \mathcal{G}$
	T_k, D_k	G_k's arrival time and deadline
	N_k, E_k	G_k's task set and edge set
	n_i^k	G_k's i-th task, $n_i^k \in N_k$
	$e_{j,i}^k$	the edge from n_j^k to n_i^k, $e_{j,i}^k \in E_k$
	$a_{j,i}^k$	the data amount from n_j^k to n_i^k
	$*n_i^k$, n_i^k*	n_i^k's immediate predecessors and successors
Resource	R	the set of PEs
	r_m	the m-th PE
	$w_{i,m}^k$	n_i^k's computation time on r_m
	$b_{m,n}$	communication speed between r_m and r_n
Scheduling	ψ	current time point
	$avail(n_i^k, r_m)$	r_m's earliest available time to execute n_i^k
	$st(n_i^k)$, $ft(n_i^k)$	n_i^k's start time and finish time
	M_k	G_k's makespan
ULS	τ_i^k, d_i^k, μ_i^k	n_i^k's dynamic top level, sub-deadline and task urgency

and execution time based on the newly introduced concept of energy delay product. Arabnejad et al. investigate how to schedule an unpredictable stream of workflows with deadline constraints on the commercial cloud [16]. A dynamic workload scheduler is devised to reduce costs. In [15], Stavrinides et al. introduce deadline constraints and simulate a heterogeneous fog environment to sort candidate tasks in a scenario where workflows arrive randomly.

We further taxonomize the above studies in Table I with additional criteria of scheduling type and multi-workflow handling manner. Specifically, for a static scheduling type, characteristics of multiple workflows are all known before scheduling. For a dynamic one, their information including graphs and arrival time can only be available on the fly when they arrive. A multi-workflow handling manner indicates whether multiple workflows are scheduled in an independent or holistic manner. In the former, workflows are scheduled independently one by one. The single workflow scheduling techniques can be adopted with slight adaptation to make use of free slots among previously scheduled tasks. Although this manner is easy to implement, existing studies [17], [18] have revealed that it underperforms for interleaved workflows. In the latter, tasks from different workflows are selected for scheduling. The common entry and common exit method is usually used to logically merge the multiple workflow graphs under consideration into one graph before scheduling [31]. Its negative effect is that the priorities and constraints of different workflows cannot be distinguished after merging.

From Table I, we can observe that as the primary scheduling concern, the workflow makespan is taken into account by all studies in the form of objectives or constraints (i.e., deadline). The energy consumption and resource rental cost are also usually considered. For the arbitrary arrival pattern, workflows arrive continuously and it may be impossible for them to be completed within their deadlines due to limited computing resources. However, how to minimize tardiness in such cases has not been investigated by existing related studies. This work substantially differs from them as it dynamically schedules deadline-constrained interleaved workflows on HCS in a holistic manner for minimizing deadline miss rate and overall tardiness.

III. PROPOSED SCHEDULING MODEL

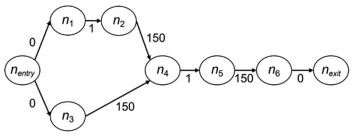
We first introduce workflow applications in HCS, then describe a scheduling framework for highly interleaved workflows, and finally formulate our concerned problem. Key notations are listed in Table II for ease of reference. Note that the workflow index is omitted when there is no ambiguity, e.g., n_i^k is shortened to n_i.

A. Workflow Applications in HCS

A complex application running on HCS can contain a number of computational tasks. The workflow model can be used to represent an application through a weighted Directed Acyclic Graph, DAG $= (N, E)$, where a node $n_i \in N$ stands for the ith task and a directed edge $e_{j,i} \in E$ stands for a precedence dependence between the jth and ith tasks. Each task n_i is indivisible and each precedence dependency $e_{j,i}$ indicates that task n_i can start executing only after n_j is completed. If $e_{j,i}$ is attached with data communication $a_{j,i}$, n_i can start only after the data from n_j has been received. The source and destination of an edge are called the parent and child tasks, respectively. The set of all n_i's direct predecessors is denoted by $*n_i$ and that of all direct successors is denoted by n_i*. To generalize the workflow with one entry and one exit, two dummy tasks n_{entry} and n_{exit} with zero execution time are added to the beginning and end of the workflow, respectively.

A heterogeneous computing system contains a certain number of heterogeneous PEs, denoted as R. When deploying a

Authorized licensed use limited to: Zhejiang University of Finance & Economics. Downloaded on August 01,2025 at 02:24:46 UTC from IEEE Xplore. Restrictions apply.



(a) DAG of the Workflow

	A7	A15	FFT
n_1	90	60	-
n_2	150	60	30
n_3	150	60	30
n_4	75	6	-
n_5	150	60	30
n_6	20	20	-

(b) Computation Time Matrix

Fig. 1. Executing a radar correlator workflow on HCS.

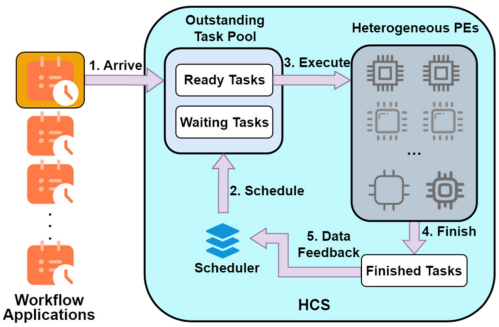


Fig. 2. Proposed scheduling framework.

workflow application onto it, each of its tasks can be executed on different PEs, and the required computation time $w_{i,m}$ of n_i on PE r_m can be obtained through analytical modeling, empirical modeling or historical data [32]. Fig. 1(a) illustrates a radar-focused workflow application named Radar Correlator [33]. It consists of 6 tasks besides the two dummy ones n_{entry} and n_{exit}, and the edge weight in the graph represents the amount of transferred data between tasks. Suppose that it is deployed on HCS consisting of an ARM Cortex-A15 core (A15), an ARM Cortex-A7 core (A7) and a Fast Fourier Transform (FFT) accelerator. Based on data profiling from [33], the execution time for each task on each PE is shown in Fig. 1(b). The required execution time of a task on different PEs varies largely. For example, the execution time of n_5 on A7 is 2.5 times that of A15 and 5 times that of the FFT accelerator.

B. Scheduling Framework for Interleaved Workflows

In practice, workflow applications can be injected to HCS at any time, i.e., new workflows may arrive before the prior ones finish. Therefore, the HCS needs to execute tasks from different workflows simultaneously. To this end, a scheduling framework, shown in Fig. 2, is proposed for HCS platforms to cope with workflows whose arrival and execution are interleaved.

When a new workflow arrives, all its tasks are injected into the outstanding task pool of the platform. Besides, there may be tasks of prior workflows still in the pool. For any workflow, tasks are ready when all their parents have been finished. Otherwise, they are held waiting in the pool.

Then, the scheduler invokes a scheduling algorithm to schedule all tasks in the pool and determine a quad $\chi_i = \langle n_i, st_i, ft_i, r_m \rangle$ for task n_i, implying that the time slice between start time st_i and finish time ft_i of PE r_m is allocated to n_i. Although tasks of prior workflows in the pool have already been scheduled, the scheduler can make more informed decisions by rescheduling them as well as the new workflow to better utilize HCS resources and enable more workflows to finish on time.

When the specific PEs become available for ready tasks, they are immediately deployed onto the PEs for execution. The information regarding task completion is fed back to the scheduler to distinguish between ready and waiting tasks.

C. Problem Formulation

For the kth workflow G_k, let T_k be its arrival time point and D_k be its deadline. That is, G_k should be finished before $D_k + T_k$. Suppose that, for G_k, its tasks n_j^k and n_i^k are respectively deployed on PEs r_n and r_m, and there is a precedence dependency $e_{j,i}^k$ with a data amount of $a_{j,i}^k$ between them. The transfer time duration $c_{j,i}^k$ of $e_{j,i}^k$ depends on $a_{j,i}^k$ and the communication speed $b_{n,m}$ between r_n and r_m. It becomes zero when r_n and r_m refer to the same PE. Hence, we have that:

$$c_{j,i}^k = \begin{cases} a_{j,i}^k / b_{n,m}, & \text{if } r_n \neq r_m, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

n_i^k can start execution on r_m when both r_m and the data from its predecessors are ready. Let $avail(n_i^k, r_m)$ be the earliest time at which PE r_m is available to execute n_i^k. Hence, the start time point and finish time point of n_i^k on r_m can be obtained as:

$$st(n_i^k) = \max \left\{ avail\left(n_i^k, r_m\right), \max_{n_j^k \in *n_i^k} \left\{ ft\left(n_j^k\right) + c_{j,i}^k \right\} \right\},$$
$$ft\left(n_i^k\right) = st\left(n_i^k\right) + w_{i,m}^k, \tag{2}$$

where $w_{i,m}^k$ represents the execution time of n_i^k on PE r_m.

Since the exit task is the last one to be executed in a workflow, the workflow makespan of G_k is obtained as,

$$M_k = ft\left(n_{exit}^k\right) - T_k. \tag{3}$$

In dynamic scenarios, workflows can arrive at the HCS unpredictably, and it is generally impossible for the scheduler to guarantee that all workflows will finish on time because of limited computing resources. Accordingly, the scheduler aims to promptly complete as many workflows as possible. When the soft deadlines of certain workflows are missed, penalties can be imposed based on the extent of their tardiness. For simplicity, we assume that all workflows have the same priority, and the penalty value of each workflow is set to equal its tardiness. Thus the other objective is to minimize the overall tardiness of all workflows.

Specifically, the value of deadline miss rate ρ is defined as,

$$\rho = \frac{\sum_{G_k \in \mathcal{G}} I(M_k > D_k)}{|\mathcal{G}|}, \tag{4}$$

where $\mathcal{G}$ is the set of all workflows injected to the HCS, and the indicator function $I(\cdot)$ returns 1 if the condition holds and otherwise 0. The value of overall tardiness η can be calculated as,

$$\eta = \sum_{G_k \in \mathcal{G}} \max(0, M_k - D_k). \tag{5}$$

Authorized licensed use limited to: Zhejiang University of Finance & Economics. Downloaded on August 01,2025 at 02:24:46 UTC from IEEE Xplore. Restrictions apply.

Thereafter, our problem can be formulated as how the scheduler performs dynamic scheduling to determine the decision variable χ for each task such that the deadline miss rate and overall tardiness are minimized.

IV. Dynamic Scheduling Algorithm ULS

When a new workflow arrives, there may be one, multiple, or no workflows currently executing in the HCS. Without loss of generality, we assume that there is one workflow G_u being executed when a new workflow, G_v, arrives at the current time point ψ, where $\psi = T_v > T_u$.[1] To try the best to ensure both G_u and G_v finish within their respective deadlines, the tasks of G_u that have not been executed are rescheduled with the tasks of G_v. A potential scheduling methodology is to combine them into a single workflow based on the technology described in [31] and then schedule it via using traditional single-workflow scheduling methods. However, this methodology cannot ensure that each workflow adheres to its deadline. List scheduling is widely used for workflow scheduling. However, existing techniques are not applicable here, because they cannot sort tasks from interleaved workflows, or co-schedule them to meet their respective deadlines.

In the proposed method ULS, we estimate the earliest start time and sub-deadline for each outstanding task in G_u and G_v by effectively capturing workflow characteristics, deadlines, and real-time execution information. Based on these factors, we then define the urgency degree of each task. Since the more urgent tasks should be scheduled and executed earlier, the two task lists from G_u and G_v are combined into an ordered list based on urgency values of tasks. By doing so, the two workflows are scheduled in an intertwined manner to adhere to their respective deadlines.

A. Task Urgency Estimation

The top level of a task n_i is widely used to statically measure the time duration from the start time of the entry task to n_i's earliest start time [34]. Since in dynamic scenarios, workflows may already be in execution, we introduce the dynamic top level τ_i to measure the time duration from the current time point ψ to n_i's earliest start time.

For the new workflow G_v, no task has started execution, and thus the dynamic top level of its task n_i^v can be obtained via

$$\tau_i^v = \max_{n_j^v \in *n_i^v} \left\{ \tau_j^v + \bar{w}_j^v + \bar{c}_{j,i}^v \right\}, \tag{6}$$

where $\bar{c}_{j,i}^v = a_{j,i}^v / \bar{b}$ is the average communication time from n_j^v to n_i^v with $\bar{b}$ being the average communication speed, and $\bar{w}_j^v$ is the average computation time of n_j^v on all resources. By traversing the workflow graph downwards from n_{entry}^v, the dynamic top level properties of all tasks in G_v can be computed recursively.

For G_u, which is currently being executed, calculating the dynamic top level property τ_i^u of its task n_i^u is more complicated, as it requires incorporating real-time execution information. For example, in Fig. 3, a Radar Correlator workflow is currently in execution, and a new workflow arrives at time point 95. At this moment, three of the prior workflow G_u's tasks have been completed, while the remaining tasks, previously assigned to PE A15, are outstanding tasks. Precisely, n_2 is sending data to n_4, and the transmission from n_3 to n_4 has finished. In this case, top levels of n_4, n_5, and n_6 should be re-calculated dynamically to capture the above real-time execution information before rescheduling.

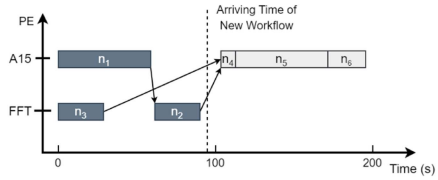


Fig. 3. An example of two workflows that are interleaved with each other.

Specifically, for an outstanding task n_i^u in G_u, the execution status of its parents may be 1) outstanding, 2) running, or 3) finished. Let W_u, X_u and F_u be the sets of currently outstanding, running and finished tasks in G_u, respectively.

If n_i^u has some outstanding parents (i.e., $*n_i^u \cap W_u \neq \emptyset$), n_i^u's end time point to receive data from them can be obtained via,

$$\phi_i(W_u) = \max_{n_j^u \in *n_i^u \cap W_u} \left\{ \tau_j^u + \bar{w}_j^u + \bar{c}_{j,i}^u \right\} + \psi. \tag{7}$$

Otherwise, $\phi_i(W_u)$ is set to ψ.

The time point that n_i^u finishes receiving data from its running parents can be obtained via,

$$\phi_i(X_u) = \begin{cases} \psi, & \text{if } *n_i^u \cap X_u = \emptyset \\ \max_{n_j^u \in *n_i^u \cap X_u} \{ ft(n_j^u) + \tilde{c}_{j,i}^u \}, & \text{otherwise,} \end{cases} \tag{8}$$

where $\tilde{c}_{j,i}^u$ is the average transfer time from n_j^u to n_i^u with the source PE determined but the destination PE unspecified.

If $*n_i^u \cap F_u \neq \emptyset$, the actual finished time (aft) of finished parents has been obtained. Since they may have transmitted or could be transmitting data to the current task n_i^u, re-scheduling it may lead to data re-transmission. For example, in Fig. 3, n_4 is initially allocated to A15 when the new workflow arrives. If it is rescheduled to be executed on a different PE, the data transmission from n_2 and n_3 to n_4 should be completely redone. During the calculation of dynamic top levels, we do not know whether n_i^u will be reallocated to the PE previously selected. Hence, we estimate this allocation consistency as a probability and calculate it via,

$$z_{j,i}^u = \frac{\tilde{c}_{j,i}^u + \bar{w}_i^u}{\max\{aft_j^u + c_{j,i}^u - \psi, 0\} + w_{i,m}^u + \tilde{c}_{j,i}^u + \bar{w}_i^u}, \tag{9}$$

where $c_{j,i}^u$ is the concrete transfer time with the source and destination PEs both specified. If the allocation consistency holds, the required time length from ψ to finish n_i^u is $\max\{aft_j^u + c_{j,i}^u - \psi, 0\} + w_{i,m}^u$, and otherwise it can be estimated as $\tilde{c}_{j,i}^u + \bar{w}_i^u$. When the ratio between them is smaller, the value of $z_{j,i}^u$ is

[1] The lexicographic order of "u" and "v" implies that G_u arrives ahead of G_v

Authorized licensed use limited to: Zhejiang University of Finance & Economics. Downloaded on August 01,2025 at 02:24:46 UTC from IEEE Xplore. Restrictions apply.

larger, i.e., closer to 1, indicating that the allocation consistency is more likely to hold.

When this consistency holds, the data arrival time from a finished parent n_j^u to n_i^u is $aft_j^u + c_{j,i}^u$, and otherwise $\psi + \tilde{c}_{j,i}^u$. Hence, the time point that n_i^u ends receiving data from its finished parents can be estimated by combining the two kinds of data arrival time based on $z_{j,i}^u$, i.e.,

$$\phi_i(F_u) = \max_{n_j^u \in *n_i^u \cap F_u} \left\{ z_{j,i}^u \left(aft_j^u + c_{j,i}^u \right) + \left(1 - z_{j,i}^u\right) \left(\psi + \tilde{c}_{j,i}^u \right) \right\}. \tag{10}$$

n_i^u's earliest start time is the time when all data from its parents have been received. By analyzing parent nodes in all statuses, n_i^u's dynamic top level, measuring the time duration from ψ to n_i^u's start time, can thus be defined as,

$$\tau_i^u = \max\left\{\phi_i(X_u), \phi_i(F_u), \phi_i(W_u)\right\} - \psi. \tag{11}$$

Similarly, by traversing the workflow graph downwards, the properties of each outstanding task in G_u can be calculated recursively.

Let $\tilde{\tau}_{exit}^k$ be the dynamic top level value of n_{exit}^k calculated based on (6), representing the time duration from n_{entry}^k's start time to n_{exit}^k's finish time. Since $\tau_i^k + \bar{w}_i^k$ measures the time length from ψ to the end of n_i^k, the sub-deadline d_i^k of node n_i^k can be defined proportionally as follows:

$$d_i^k = \frac{\tau_i^k + \bar{w}_i^k}{\tilde{\tau}_{exit}^k} \times \left(D_k + T_k - \psi\right). \tag{12}$$

In (12), instead of $\tilde{\tau}_{exit}^k$, an intuitive alternative to define the sub-deadline is to use τ_{exit}^k calculated via (11) as the denominator, which measures the length from ψ to n_{exit}^k's finish time. For the new workflow G_v, $\tau_{exit}^v = \tilde{\tau}_{exit}^v$ because $\psi = T_v$; while for the prior one G_u, $\tau_{exit}^u < \tilde{\tau}_{exit}^u$ because some tasks may have already finished. When using τ_{exit}^k, sub-deadlines of outstanding tasks in both new and prior workflows are set uniformly. By contrast, when using $\tilde{\tau}_{exit}^k$, the sub-deadlines for G_u are set in a more stringent way than that for G_v. When a greater proportion of G_u has been finished, this stringency is higher. To schedule high-interleaved workflows, it is reasonable to prefer executing tasks of prior workflows to some extent because they are closer to their deadlines at the moment. Based on this consideration, $\tilde{\tau}_{exit}^k$ is used in (12).

The urgency of a task is manifested in its earliest start time and latest finish time, which are measured by the dynamic top level τ and sub-deadline d, respectively. The former is vital to make more tasks in the workflow ready, and the latter directly determines whether the deadline of the workflow can be met or not. Hence, based on a weight parameter $\omega \in [0, 1]$, the urgency of a task is defined as,

$$\mu_i^k = \omega \times d_i^k + (1 - \omega) \times \tau_i^k. \tag{13}$$

A lower μ_i^k value indicates that task n_i^k is more urgent and should be allocated to a PE earlier. Note that given any two outstanding tasks n_i^k and n_j^k in graph G_k with $n_i^k \in *n_j^k$, $\tau_i^k < \tau_j^k$ and $d_i^k < d_j^k$ based on the definitions of τ and d. Therefore, μ_i^k is always smaller than μ_j^k, and in the task list yielded by task ranking according to urgency values increasingly, all the precedence dependencies among tasks in the workflow graphs are respected, which is a precondition to conduct a list scheduling.

Algorithm 1: ULS.

```
   Input: Currently unfinished workflows
1  L ← [];
2  for G_k in unfinished workflows do
3      if T_k + D_k < ψ then
4          continue;
5      for each outstanding task n_i^k of G_k do
6          calculate τ_i^k via (6) or (11);
7          L.append(n_i^k);
8      end
9  end
10 for each task n_i^k in L do
11     calculate d_i^k via (12) and μ_i^k via (13);
12 end
13 sort L in an ascending order of μ;
14 for each task n_i^k in L do
15     best ←< n_i^k, +∞, +∞, NULL >;
16     for PE r_m in R do
17         χ ← invoke Algo. 2 with n_i^k, r_m as input;
18         if χ.ft < best.ft then
19             best ← χ;
20     end
21     assign n_i^k according to best;
22 end
```

B. Implementation

The proposed ULS algorithm is realized in Algorithm 1. Its input is the currently unfinished workflows, including newly injected workflows and those being executed. The algorithm consists of two phases: task prioritizing (Lines 1-13) and PE selection (Lines 14-22). In the first phase, the scheduler initially checks whether each unfinished workflow G_k in the framework has expired (Lines 3-4), as some prior workflows may have already timed out. In such cases, we skip the workflow for rescheduling, and continue using its prior scheduling solution to avoid introducing additional tardiness. For the remaining workflows, all their outstanding tasks—specifically those in a ready or waiting state, as illustrated in Fig. 2—are added to list L, and their urgency values are computed (Lines 5-12). For newly injected workflows, (6) is used, while (11) is applied for others. The combined task list L is then sorted based on such values (Line 13).

In the second phase, the scheduler sequentially traverses the tasks in L. For each task, it tests all candidate resources via tentative allocations to find the one that allows for the earliest finish time (Lines 15-20) and assigns the task to it (Line 21).

The detail of allocating n_i^k to r_m tentatively is elaborated in Algorithm 2. Specifically, if G_k is not a new workflow (Line 2), we first obtain PE r_n from its previous scheduling solution where n_i^k is allocated (Line 3). The time point when n_i^k ends

Authorized licensed use limited to: Zhejiang University of Finance & Economics. Downloaded on August 01,2025 at 02:24:46 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Tentative_Task_Allocation.

Input: n_i^k,r_m
Output: χi
1 $\phi_{max} \leftarrow 0$, $r_n \leftarrow NULL$;
2 **if** $T_k < \psi$ **then**
3 $\quad r_n \leftarrow$ the PE where n_i^k was previously allocated;
4 **for** *each task* n_j^k *in* $*n_i^k$ **do**
5 $\quad$ **if** $n_j^k \in F_k$ *and* $r_m = r_n$ **then**
6 $\quad\quad \phi_{j,i} \leftarrow aft_j^k + c_{j,i}^k$;
7 $\quad$ **else if** $n_j^k \in F_k$ *and* $r_m \neq r_n$ **then**
8 $\quad\quad \phi_{j,i} \leftarrow \psi + \tilde{c}_{j,i}^k$;
9 $\quad$ **else if** $n_j^k \in X_k$ **then**
10 $\quad\quad \phi_{j,i} \leftarrow ft(n_j^k) + \tilde{c}_{j,i}^k$;
11 $\quad$ **else**
12 $\quad\quad \phi_{j,i} \leftarrow ft(n_j^k) + c_{j,i}^k$;
13 $\quad \phi_{max} \leftarrow \max\{\phi_{max}, \phi_{j,i}\}$;
14 **end**
15 $avail(n_i^k, r_m) \leftarrow$ search the ready time of r_m for n_i^k;
16 $st(n_i^k) \leftarrow \max\{\phi_{max}, avail(n_i^k, r_m)\}$;
17 return $\langle n_i^k, st(n_i^k), st(n_i^k) + w_{i,m}^k, r_m \rangle$;

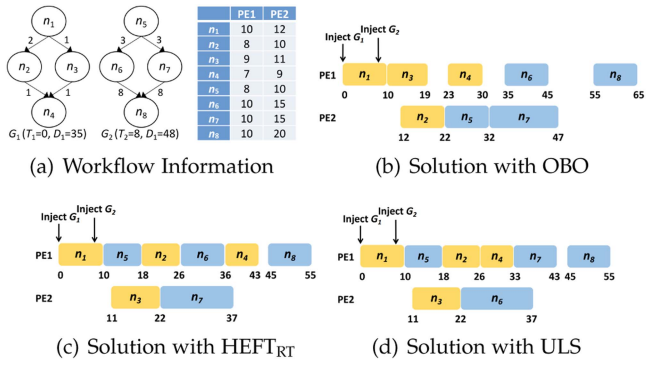
receiving data from its parent n_j^k, denoted as $\phi_{j,i}$, depends on n_j^k's execution status and whether the allocation consistency holds (Lines 5-12). Moreover, the earliest ready time of r_m for n_i^k is searched with an insertion policy, i.e., the task is inserted between already scheduled tasks in r_m if the idle time slot between them is enough for executing n_i^k. Afterward, the scheduling detail of this tentative allocation is determined and returned (Lines 16-17).

In the first phase of ULS, all tasks and edges in each considered workflow are traversed for calculating task urgency values, and the merged task list is sorted. Suppose that there are at most k workflows to be scheduled simultaneously, with at most n tasks in each workflow. The maximum number of edge dependencies in a workflow is $n(n-1)/2$, and therefore, the complexity of this phase is $O(kn^2 + kn\log_2(kn))$. It can be reduced to $O(kn^2)$ as in general $n > \log_2(kn)$. In the task allocation step, l candidate PE resources are traversed for each of the kn tasks, and the allocation function in Algorithm 2 is invoked. This function traverses all parents of a task, whose upper bound is n. Hence, the time complexity of the second phase is $O(kn^2 l)$. Overall, the time complexity of ULS is $O(kn^2 l)$.

C. Illustrative Example

Fig. 4(a) shows two sample workflows G_1 and G_2 with the transfer time attached on the edges, as well as the computation time of all eight tasks on two PEs. Suppose that G_1 arriving at time 0 has a deadline of 35 whereas G_2 arriving at time 8 should finish before 56, i.e., $D_2 = 56 - T_2 = 48$.

We first try to schedule them one by one (OBO). The most widely used single workflow scheduling heuristic HEFT [19] is used here after being adapted to make use of free slots among previously scheduled workflows. The scheduling result is shown



(a) Workflow Information (b) Solution with OBO

(c) Solution with HEFT_{RT} (d) Solution with ULS

Fig. 4. Scheduling interleaved workflows with different methods.

TABLE III
SCHEDULING DETAILS WITH THE ULS ALGORITHM

Time Point 0				
Node	n_1	n_2	n_3	n_4
τ	0	13	12	23
d	12.42	24.84	24.84	35
μ	6.21	18.92	18.42	29

Time Point 8								
Node	n_1	n_2	n_3	n_4	n_5	n_6	n_7	n_8
τ	\	4	3	14	0	12	12	32.5
d	\	11.32	11.32	19.16	9.09	24.76	24.76	48
μ	\	7.66	7.16	16.58	4.55	18.38	18.38	40.25

in Fig. 4(b): although G_1 finishes on time, G_2 expires with a tardiness of 9.

Next, we resort to the recently proposed efficient multi-workflow scheduling method HEFT_{RT} [17], which aims to minimize the makespan of all workflows. The result is shown in Fig. 4(c). In this case, although G_2 finishes on time, G_1 expires with a tardiness of 8.

At last, the proposed method ULS is used. Table III lists the τ, d and μ values of each task for both moments when G_1 and G_2 are injected and ULS is invoked. The scheduling result is shown in Fig. 4(d), where both workflows finish within their respective deadlines.

The OBO and HEFT_{RT} methods underperform here because OBO does not reschedule G_1 to help G_2 have a better schedule, and HEFT_{RT} cannot distinguish between task urgency for adhering to deadlines during rescheduling. By comparison, ULS effectively recognizes urgency degrees of outstanding tasks from G_1 and G_2, and is thus able to make both of them finish on time. This example highlights the significant superiority of ULS over its two peers.

V. EXPERIMENTAL EVALUATIONS

This section reports the experiments conducted to validate the proposed method and discusses the key findings.

A. Experimental Settings

DS3[2] is an open-source high-level HCS simulator specially designed for the evaluation of scheduling and power management algorithms for different hardware configurations [33]. It

[2][Online]. Available: https://github.com/segemena/DS3

Authorized licensed use limited to: Zhejiang University of Finance & Economics. Downloaded on August 01,2025 at 02:24:46 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
TWO HCS CONFIGURATIONS IN DS3 USED FOR EXPERIMENTAL EVALUATIONS

	Config. I	Config. II
General-Purpose PE	4 x A7 4 x A15	4 x A53 4 x A72
Hardware Accelerator	2 x MM 4 x FFT 2 x VD	2 x MM 4 x FFT 2 x VD 2 x SCE

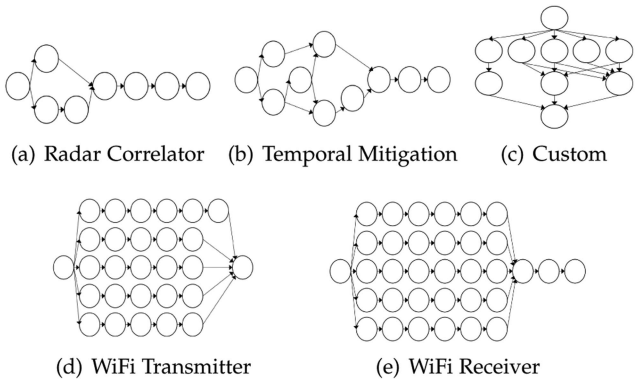


Fig. 5. The structures of four real-world workflows and one custom workflow in DS3.

has been validated against real-world System-on-Chip platforms based on various configurations and applications, and the findings reveal that the average difference between the simulation results and reference realistic measurements is less than 3%. Based on DS3, we implement the proposed scheduling framework.[3] Evaluation experiments are conducted on a PC with 12th Gen Intel Core i7-12700H, Windows 11.

As shown in Table IV, two configurations of HCS platforms available in DS3 are used: Config. I and Config. II. The former consists of 8 general-purpose PEs: 4 ARM Cortex-A7 cores and 4 ARM Cortex-A15 cores, and is equipped with 2 Matrix Multiply (MM) accelerators, 4 FFT accelerators, and 2 Viterbi Decoders (VD) accelerators. In the latter, the ARM Cortex-A53 and Cortex-A72 cores, which have greater capabilities than A7 and A15, are used as the general-purpose PEs, and 2 additional scrambler_encoder (SCE) accelerators are equipped. Thus, Config. II is more powerful than Config. I.

A workload mixture is used for evaluation which consists of five workflow applications available in DS3, i.e., Radar Correlator, Temporal Mitigation, WiFi Transmitter, WiFi Receiver, and a custom workflow. Their structures are shown in Fig. 5 and detailed characteristics are listed in Table V. The computation time of each task on each kind of PE, and the transferred data amount among tasks are all available in DS3 [33].

The workflows are injected into the HCS framework continually and for each injection one of the five workflows is randomly used. The injection intervals between two consecutive workflows are assumed to follow an exponential distribution Exp(λ) just as in [13], [14], [17], [18]. To be more precise, the parameter λ represents the expected number of workflow arrivals in one second, and $\frac{1}{\lambda}$ represents the expected arrival interval between two consecutive workflows. A larger λ value indicates a higher workload on the considered HCS platform, and a larger scale of interleaved workflows to be scheduled. To evaluate the effectiveness of a scheduling method in handling a high workload, new workflows are admitted even when the platform is overloaded.

[3][Online]. Available: https://github.com/im-Goodman/ULS

TABLE V
THE DETAILED CHARACTERISTICS OF THE USED FIVE WORKFLOWS

Application	Max Width	Depth	Task Number	Supported Accelerators
Radar Correlator	2	7	8	FFT
Temporal Mitigation	2	8	10	Matrix Multiply
WiFi Transmitter	5	8	28	IFFT
WiFi Receiver	5	10	34	FFT and Viterbi Decoder
Custom Workflow	5	4	10	IFFT

TABLE VI
THREE TYPES OF DEADLINE SETTINGS

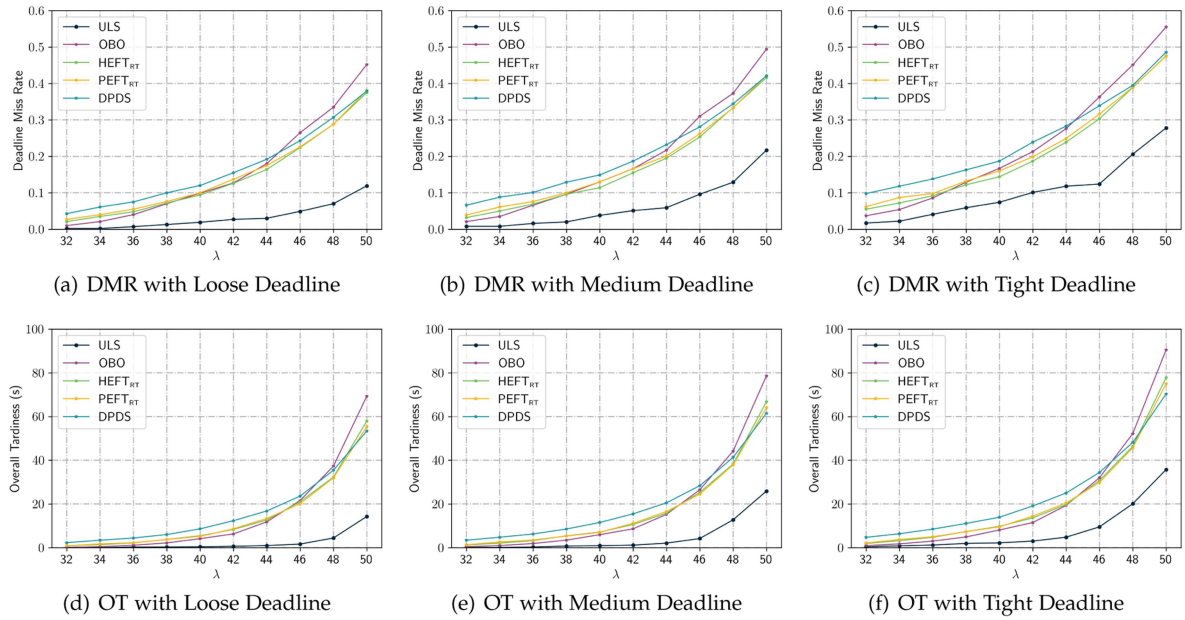
Type	$2 \times M$	$3 \times M$
Tight	80%	20%
Medium	50%	50%
Loose	20%	80%

To set the deadline for each workflow on the HCS platform, we first schedule it with HEFT and execute it to record its makespan (denoted as M). As shown in Table VI, we introduce three types of deadlines based on M: tight, medium, and loose. For example, for the tight one, the deadline of a workflow is set to 2 times its makespan in a 80% probability, and 3 times in a 20% probability.

1) Compared Methods and Evaluation Metrics: The following state-of-the-art methods are used for comparison of the proposed methods: OBO, HEFT_{RT}, PEFT_{RT} and DPDS (Dynamic Probabilistic Deadline-Aware Scheduling). Specifically, in OBO, each workflow is scheduled by HEFT [19] based on the first-come-first-service (FCFS) policy, and free slots among tasks of prior workflows are exploited when scheduling a new one, as shown in Section IV-C. Although there are many single workflow scheduling methods proposed recently (e.g., [35], [36]), they are quite similar to HEFT in the considered model, especially on resource contention handling, and hence we adopt HEFT in OBO which is the most classic. HEFT_{RT} is a multi-workflow scheduling methods from [17] which aims to minimize the makespan of each workflow. PEFT_{RT} is a variant of the single workflow scheduling method PEFT [20] for multiple workflows, and it computes an optimal cost table of unfinished tasks each time for task scheduling. DPDS [13] conducts a dynamic multi-workflow scheduling to minimize the job-level risk under deadline constraints. When all the PEs are safe like in our experiments, it assigns a ready task to PE that allows the earliest finish time. For the proposed method ULS, the parameter ω is set to 0.5 based on parameter tuning.

A total of 1000 instances of the five used workflows are randomly injected for experimental evaluations. Each time a workflow instance arrives, the scheduling algorithm generates a schedule for it and outstanding tasks of prior workflows. Using this schedule as input, DS3 simulates the execution of workflow instances and outputs their makespan values. We use

Authorized licensed use limited to: Zhejiang University of Finance & Economics. Downloaded on August 01,2025 at 02:24:46 UTC from IEEE Xplore. Restrictions apply.



(a) DMR with Loose Deadline (b) DMR with Medium Deadline (c) DMR with Tight Deadline

(d) OT with Loose Deadline (e) OT with Medium Deadline (f) OT with Tight Deadline

Fig. 6. The DMR and OT values of each method w.r.t. λ with Config. I.

three metrics to evaluate the performance of different scheduling algorithms: Deadline Miss Rate (DMR), Overall Tardiness (OT), and Average Concurrent Jobs (ACJ). ACJ is the average number of unfinished workflows in the HCS when a new workflow arrives.

B. Experimental Results and Analysis

To evaluate the ability of each method to obtain a valid solution, we gradually vary the parameter λ from 32 to 50 in a step size of 2 with Config. I under the three deadline settings. The corresponding DMR values of each approach are depicted in Fig. 6(a)–(c). On the one side, when λ becomes larger (i.e., the workload is higher), the DMR values of all compared methods rise because more workflows can not be finished on time. On the other side, when the deadline becomes tighter, more workflows expire. For example, as shown in Fig. 6(a)–(c), in the case of $\lambda = 50$, ULS's DMR is 0.12 for the loose deadline setting, and it turns to 0.22 and 0.28 for the medium and tight settings, respectively.

The DMR value of ULS is significantly lower than those of its peers in all cases. For example, when $\lambda = 50$ with the loose deadline, the DMR of ULS is 0.12 whereas those of DPDS, OBO, HEFT$_{RT}$ and PEFT$_{RT}$ are 0.38, 0.45, 0.38 and 0.38, respectively, much higher than ULS's. On average, ULS reduces the DMR value by 74.0%, 68.8%, 68.1%, and 70.0% against DPDS, OBO, HEFT$_{RT}$ and PEFT$_{RT}$, respectively.

When the workload is light, the single-workflow scheduling method OBO outperforms HEFT$_{RT}$ and PEFT$_{RT}$ due to its FCFS scheduling policy. On the other side, under heavy loads, these two multi-workflow scheduling methods achieve a lower DMR value than OBO. DPDS performs the worst when $\lambda < 44$ and is a bit better than OBO when λ is very large.

Fig. 6(d)–(f) depicts the corresponding OT value of each method. Similarly, with a greater λ value and a tighter deadline setting, the OT value of each method becomes higher and ULS beats its peers in all cases. For example, with the loose deadline setting, when $\lambda < 40$, the OT value of ULS is close to 0, which is almost negligible compared with the others. On average, ULS can reduce OT by 85.5%, 77.2%, 81.6% and 81.8% against DPDS, OBO, HEFT$_{RT}$ and PEFT$_{RT}$, respectively.

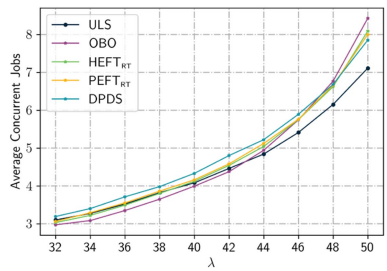


Fig. 7. The ACJ values of each method with medium deadline type and Conf. I.

The ACJ result of each method with the medium deadline setting is shown in Fig. 7. The ACJ value rises with the increase of λ because the injection interval becomes small. When $\lambda \leq 42$, ULS's ACJ value is larger than OBO's, less than DPDS's, and similar to HEFT$_{RT}$ and PEFT$_{RT}$. For each method, a low ACJ value does not necessarily correspond to low DMR and OT values, though they are related. For example, OBO, owing to its FCFS scheduling policy, has the fewest workflows backlogged and hence achieves the lowest ACJ value, although it underperforms in terms of DMR and OT. DPDS's scheduling policy causes a substantial backlog of unfinished workflows, resulting in a high ACJ value. For the multi-workflow scheduling methods ULS, HEFT$_{RT}$ and PEFT$_{RT}$, their ACJ values are very close under a loose workload condition.

When $\lambda \geq 44$, ULS's ACJ value is the smallest among all. This implies that when the system workload is high, ULS

Authorized licensed use limited to: Zhejiang University of Finance & Economics. Downloaded on August 01,2025 at 02:24:46 UTC from IEEE Xplore. Restrictions apply.

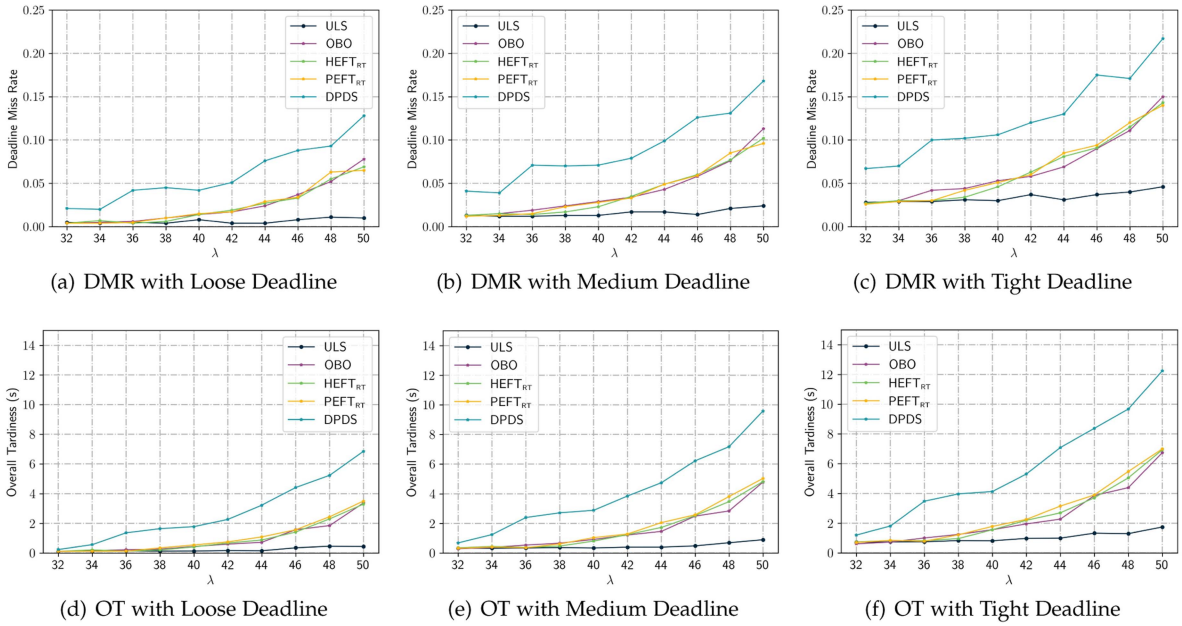


Fig. 8. The DMR and OT values of each method w.r.t. λ with Config. II.

can schedule interleaved workflows more effectively and fewer workflows are backlogged in the platform. Meanwhile, we can observe that OBO's ACJ value rises very quickly and becomes the largest when $\lambda \geq 48$, implying that it is ineffective to schedule highly interleaved workflows by using an FCFS policy. The ACJ results for the loose and tight deadline settings are similar and are thus not included in the paper.

Afterwards, the experiment of scheduling the workload mixture including 1000 workflow instances is carried out with the Config. II of HCS, and the results are depicted in Fig. 8. We can observe that by using more powerful PEs, the DMR and OT values of each method in each case is much lower than that with Config. I. For example, when the deadline setting is tight and $\lambda = 50$, the DMR value of ULS is close to 0.3 in Config. I, and it declines to 0.05 in Config. II. Nevertheless, for both metrics, ULS performs the best among all compared methods.

At last, the runtime for each method to schedule workflows is compared. Specifically, a workload consisting of a mixture of the five workflows is used and their deadlines are set to $2\times M$ or $3\times M$ randomly. A total of 100 instances of these workflows are injected to HCS with Configs. I and II, respectively. For each scheduling method, the runtime overhead of each invocation is recorded. Note that ULS and OBO are invoked each time when a new workflow arrives whereas HEFT$_{RT}$, PEFT$_{RT}$ and DPDS are invoked when certain tasks turn ready. Hence, Table VII lists the number of invocations for each method, the total runtime to schedule all the workflows, and the mean runtime per invocation.

From Table VII, we can observe that the invocation count for ULS and OBO is 100, which matches the number of injected workflows. By contrast, the invocation numbers for HEFT$_{RT}$, PEFT$_{RT}$, and DPDS are significantly higher, and slightly differ from each other because of their respective scheduling strategies. HEFT$_{RT}$ incurs the least total runtime as it only schedules ready tasks and does not need to handle precedence dependencies

TABLE VII
RUNTIME (μS) OF THE COMPARED ALGORITHMS

Config. I			
Algorithm	Invocation No.	Total Runtime	Mean Runtime
ULS	100	4284	42.84
OBO	100	5479	54.79
HEFT$_{RT}$	1226	3205	2.61
PEFT$_{RT}$	1247	36772	29.49
DPDS	1258	29056	23.10
Config. II			
Algorithm	Invocation No.	Total Runtime	Mean Runtime
ULS	100	5467	54.67
OBO	100	5745	57.45
HEFT$_{RT}$	1200	3059	2.55
PEFT$_{RT}$	1267	46599	36.78
DPDS	1241	27261	21.97

between tasks. OBO's total runtime is almost twice that of HEFT$_{RT}$'s. PEFT$_{RT}$ requires the most runtime as it needs to compute an optimal cost table for each invocation. The proposed method ULS incurs more runtime than HEFT$_{RT}$ as it reschedules certain outstanding tasks. ULS consumes an average of 42.84μs for each invocation with Config. I, and 54.67μs with Config. II. The runtime overhead is negligible compared with the overall tardiness, which are in the order of seconds as shown in Figs. 6 and 8, and hence is applicable to real-time scheduling scenarios. Note that if the scheduler is implemented in a hardware-based way, the runtime overhead can be greatly reduced. For example, in [37], the hardware-based HEFT$_{RT}$ method is demonstrated to make scheduling decisions with 10ns latency on average.

VI. CONCLUSION AND FUTURE WORK

In this paper, we have proposed an interleaved multi-workflow scheduling algorithm named ULS to minimize the deadline miss rate and overall tardiness for deadline-constrained workflows

Authorized licensed use limited to: Zhejiang University of Finance & Economics. Downloaded on August 01,2025 at 02:24:46 UTC from IEEE Xplore. Restrictions apply.

that arrive dynamically. To make workflows finish on time as much as possible, it schedules each newly arriving workflow as well as the outstanding tasks of prior workflows. A novel task status indicator called task urgency is for the first time proposed to prioritize tasks from different workflows and all outstanding tasks are scheduled in a list-based way. The evaluation experiments are conducted on simulated HCSs. Results demonstrate that ULS can reduce at least 68% of DMR and 77% of OT on average compared with state-of-the-art scheduling methods.

Our future work intends to delve into theoretical analysis of workflow scheduling and the proposed technique, while extending it to incorporate additional objectives such as energy consumption [10], [38], [39], [40], rental cost [7], [24], and fault tolerance [13], [41] in HCS. We are also interested in effectively scheduling interleaved workflows with varying priorities and complex penalty policies.

Acknowledgments

The authors would like to thank the editor and anonymous reviewers for their valuable comments/suggestions that help us improve this article.

References

[1] C. Hagleitner et al., "Heterogeneous computing systems for complex scientific discovery workflows," in *Proc. 2021 Des. Autom. Test Europe Conf. Exhib.*, 2021, pp. 13–18.
[2] X. Liu, R. Wang, C. Shi, C. Zou, and W. Zhu, "Computing acceleration to genome-wide association study based on CPU/FPGA heterogeneous system," *ACM SIGAPP Appl. Comput. Rev.*, vol. 23, no. 4, pp. 16–26, 2024.
[3] W. Tan and M. Zhou, *Business and Scientific Workflows: A Web Service-Oriented Approach*. Hoboken, NJ, USA: Wiley, 2013.
[4] Z. Sun, B. Zhang, C. Gu, R. Xie, B. Qian, and H. Huang, "ET2FA: A hybrid heuristic algorithm for deadline-constrained workflow scheduling in cloud," *IEEE Trans. Serv. Comput.*, vol. 16, no. 3, pp. 1807–1821, May/Jun. 2023.
[5] X. Li, M. A. Salehi, M. Bayoumi, N.-F. Tzeng, and R. Buyya, "Cost-efficient and robust on-demand video transcoding using heterogeneous cloud services," *IEEE Trans. Parallel Distrib. Syst.*, vol. 29, no. 3, pp. 556–571, Mar. 2018.
[6] L. Versluis and A. Iosup, "A survey of domains in workflow scheduling in computing infrastructures: Community and keyword analysis, emerging trends, and taxonomies," *Future Gener. Comput. Syst.*, vol. 123, pp. 156–177, 2021.
[7] X. Wang, J. Cao, and R. Buyya, "Adaptive cloud bundle provisioning and multi-workflow scheduling via coalition reinforcement learning," *IEEE Trans. Comput.*, vol. 72, no. 4, pp. 1041–1054, Apr. 2023.
[8] H. Li, D. Wang, M. Zhou, Y. Fan, and Y. Xia, "Multi-swarm co-evolution based hybrid intelligent optimization for bi-objective multi-workflow scheduling in the cloud," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 9, pp. 2183–2197, Sep. 2022.
[9] M. Malawski, G. Juve, E. Deelman, and J. Nabrzyski, "Algorithms for cost- and deadline-constrained provisioning for scientific workflow ensembles in IaaS clouds," *Future Gener. Comput. Syst.*, vol. 48, pp. 1–18, 2015.
[10] D. Senapati, A. Sarkar, and C. Karfa, "Energy-aware real-time scheduling of multiple periodic dags on heterogeneous systems," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 42, no. 8, pp. 2447–2460, Aug. 2023.
[11] S. K. Roy, R. Devaraj, and A. Sarkar, "SAFLA: Scheduling multiple real-time periodic task graphs on heterogeneous systems," *IEEE Trans. Comput.*, vol. 72, no. 4, pp. 1067–1080, Apr. 2023.
[12] H. Taheri, S. Abrishami, and M. Naghibzadeh, "A cloud broker for executing deadline-constrained periodic scientific workflows," *IEEE Trans. Serv. Comput.*, vol. 16, no. 5, pp. 3089–3100, Sep./Oct. 2023.
[13] X. Chen, U. Ogras, and C. Chakrabarti, "Probabilistic risk-aware scheduling with deadline constraint for heterogeneous SoCs," *ACM Trans. Embed. Comput. Syst.*, vol. 21, no. 2, pp. 1–27, 2022.
[14] X. Ma, H. Xu, H. Gao, and M. Bian, "Real-time multiple-workflow scheduling in cloud environments," *IEEE Trans. Netw. Serv. Manag.*, vol. 18, no. 4, pp. 4002–4018, Dec. 2021.
[15] G. L. Stavrinides and H. D. Karatza, "Orchestration of real-time workflows with varying input data locality in a heterogeneous fog environment," in *Proc. 5th Int. Conf. Fog Mobile Edge Comput.*, 2020, pp. 202–209.
[16] V. Arabnejad, K. Bubendorfer, and B. Ng, "Dynamic multi-workflow scheduling: A deadline and cost-aware approach for commercial clouds," *Future Gener. Comput. Syst.*, vol. 100, pp. 98–108, 2019.
[17] J. Mack, S. E. Arda, U. Y. Ogras, and A. Akoglu, "Performant, multi-objective scheduling of highly interleaved task graphs on heterogeneous system on chip devices," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 9, pp. 2148–2162, Sep. 2022.
[18] L. F. Bittencourt and E. R. Madeira, "Towards the scheduling of multiple workflows on computational grids," *J. Grid Comput.*, vol. 8, no. 3, pp. 419–441, 2010.
[19] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.
[20] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 3, pp. 682–694, Mar. 2014.
[21] H. Wang and O. Sinnen, "List-scheduling versus cluster-scheduling," *IEEE Trans. Parallel Distrib. Syst.*, vol. 29, no. 8, pp. 1736–1749, Aug. 2018.
[22] M. Y. Özkaya, A. Benoit, B. Uçar, J. Herrmann, and Ü. V. Çatalyürek, "A scalable clustering-based task scheduler for homogeneous processors using DAG partitioning," in *Proc. 2019 IEEE Int. Parallel Distrib. Process. Symp.*, 2019, pp. 155–165.
[23] X. Tang, K. Li, G. Liao, and R. Li, "List scheduling with duplication for heterogeneous computing systems," *J. Parallel Distrib. Comput.*, vol. 70, no. 4, pp. 323–329, 2010.
[24] H. Li, B. Wang, Y. Yuan, M. Zhou, Y. Fan, and Y. Xia, "Scoring and dynamic hierarchy-based NSGA-II for multiobjective workflow scheduling in the cloud," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 2, pp. 982–993, Apr. 2022.
[25] L. Yin, J. Sun, J. Zhou, Z. Gu, and K. Li, "ECFA: An efficient convergent firefly algorithm for solving task scheduling problems in cloud-edge computing," *IEEE Trans. Serv. Comput.*, vol. 16, no. 5, pp. 3280–3293, Sep./Oct. 2023.
[26] M. Xu et al., "Genetic programming for dynamic workflow scheduling in fog computing," *IEEE Trans. Serv. Comput.*, vol. 16, no. 4, pp. 2657–2671, Jul./Aug. 2023.
[27] M. H. Hilman, M. A. Rodriguez, and R. Buyya, "Multiple workflows scheduling in multi-tenant distributed systems: A taxonomy and future directions," *ACM Comput. Surv.*, vol. 53, no. 1, pp. 1–39, 2020.
[28] H. Arabnejad and J. G. Barbosa, "Multi-workflow QoS-constrained scheduling for utility computing," in *Proc. IEEE 18th Int. Conf. Comput. Sci. Eng.*, 2015, pp. 137–144.
[29] G. L. Stavrinides and H. D. Karatza, "The impact of input error on the scheduling of task graphs with imprecise computations in heterogeneous distributed real-time systems," in *Proc. 18th Int. Conf. Anal. Stochastic Model. Techn. Appl.*, Venice, Italy, Springer, 2011, pp. 273–287.
[30] S. Zhao, X. Dai, and I. Bate, "DAG scheduling and analysis on multi-core systems by modelling parallelism and dependency," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4019–4038, Dec. 2022.
[31] H. Zhao and R. Sakellariou, "Scheduling multiple DAGs onto heterogeneous systems," in *Proc. 20th IEEE Int. Parallel Distrib. Process. Symp.*, 2006, pp. 14–pp.
[32] R. Wilhelm et al., "The worst-case execution-time problem–overview of methods and survey of tools," *ACM Trans. Embeded. Comput. Syst.*, vol. 7, no. 3, pp. 1–53, 2008.
[33] S. E. Arda et al., "DS3: A system-level domain-specific system-on-chip simulation framework," *IEEE Trans. Comput.*, vol. 69, no. 8, pp. 1248–1262, Aug. 2020.
[34] Y.-K. Kwok and I. Ahmad, "Static scheduling algorithms for allocating directed task graphs to multiprocessors," *ACM Comput. Surv.*, vol. 31, no. 4, pp. 406–471, 1999.
[35] D. Senapati, A. Sarkar, and C. Karfa, "HMDS: A makespan minimizing DAG scheduler for heterogeneous distributed systems," *ACM Trans. Embed. Comput. Syst.*, vol. 20, no. 5s, pp. 1–26, 2021.
[36] H. Djigal, J. Feng, J. Lu, and J. Ge, "IPPTS: An efficient algorithm for scientific workflow scheduling in heterogeneous computing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1057–1071, May 2021.

Authorized licensed use limited to: Zhejiang University of Finance & Economics. Downloaded on August 01,2025 at 02:24:46 UTC from IEEE Xplore. Restrictions apply.

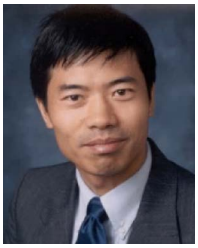
[37] A. Fusco, S. Hassan, J. Mack, and A. Akoglu, "A hardware-based heft scheduler implementation for dynamic workloads on heterogeneous SoCs," in *Proc. IFIP/IEEE 30th Int. Conf. Very Large Scale Integr.*, 2022, pp. 1–6.
[38] B. Hu, Z. Cao, and M. Zhou, "Energy-minimized scheduling of real-time parallel workflows on heterogeneous distributed computing systems," *IEEE Trans. Serv. Comput.*, vol. 15, no. 5, pp. 2766–2779, Sep./Oct. 2022.
[39] H. Yuan, J. Bi, J. Zhang, and M. Zhou, "Energy consumption and performance optimized task scheduling in distributed data centers," *IEEE Trans. Syst. Man Cybern. Syst.*, vol. 52, no. 9, pp. 5506–5517, Sep. 2022.
[40] B. Hu, Y. Shi, G. Chen, Z. Cao, and M. Zhou, "Workload-aware scheduling of real-time jobs in cloud computing to minimize energy consumption," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 638–652, Jan. 2024.
[41] P. Zhang et al., "A fault-tolerant model for performance optimization of a fog computing system," *IEEE Internet Things J.*, vol. 9, no. 3, pp. 1725–1736, Feb. 2022.



Kun Cai received the BS degree from the Chongqing University of Computer Science, Chongqing, China, in 2022. He is currently working toward the master's degree in computer technology with Chongqing University. His main research interests include workflow scheduling and heterogeneous computing systems.



Quanwang Wu (Senior Member, IEEE) received the BS, MS, and PhD degrees in computer science from Chongqing University, in 2007, 2010, and 2013, respectively. He is currently an associate professor of the School of Computer Science in Chongqing University, Chongqing, China. He was a special researcher with the digital content and media sciences research division of the National Institute of Informatics (NII) in Tokyo, Japan from 2014 to 2015. His main research interests include scheduling techniques, services computing, and data mining.



Mengchu Zhou (Fellow, IEEE) received the PhD degree from Rensselaer Polytechnic Institute, Troy, New York, in 1990 and then joined New Jersey Institute of Technology where he has been distinguished professor since 2013. His interests are in Petri nets, automation, robotics, big data, Internet of Things, cloud/edge computing, and AI. He has more than 1200 publications including 17 books, more than 850 journal papers (more than 650 in IEEE transactions), 31 patents and 32 book-chapters. He is fellow of IFAC, AAAS, CAA and NAI.



Chao Chen (Senior Member, IEEE) received the BS and MS degrees in control science and engineering from Northwestern Polytechnical University, Xi'an, China, in 2007 and 2010, respectively, and the joint PhD degree from Sorbonne University and Institut Mines-Télécom/Télécom SudParis, France, in 2014. He is currently a full professor with the College of Computer Science, Chongqing University, Chongqing, China. He has published more than 100 papers, including 40 ACM/IEEE Transactions. His research interests include pervasive computing, mobile computing, urban logistics, data mining from large-scale GPS trajectory data, and big data analytics for smart cities.



Junhao Wen received the PhD degree from Chongqing University, in 2008, where he is a professor with the College of Big Data and Software Engineering. His research interests include service computing, cloud computing, and software dependable engineering. He has published more than 100 refereed journal and conference papers. He has more than 30 projects and developed many commercial systems and software tools.



Shouguang Wang received the BS degree from the Changsha University of Science and Technology, and the PhD degree from Zhejiang University. He joined Zhejiang Gongshang University, in 2005, where he is currently a professor and dean of System Modeling and Control Research Institute. He was a visiting professor with the New Jersey Institute of Technology, and University of Cagliari, Italy. He is associate editor of *IEEE/CAA Journal of Automatica Sinica*. He has many publications including more than 40 peer-review journal papers. His interests are in Petri net theory and application, scheduling, optimization and discrete event systems.

Authorized licensed use limited to: Zhejiang University of Finance & Economics. Downloaded on August 01,2025 at 02:24:46 UTC from IEEE Xplore. Restrictions apply.